SOFTWARE TESTING

LAB RECORD – QUESTIONS 6 TO 30

Name: D.J Alan Prince

Register Number: 1911260303

Course Name: Software Testing

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Questions are arranged continuously to use page space efficiently.

PART A – CASE STUDIES AND TEST PLANS (Q6–Q15)

QUESTION 6: Food Delivery Application – Positive Test Scenarios

Requirement analysis: Identify requirements for customer login, delivery address, restaurant listing, menu, cart, payment and order tracking.

Test scenarios and expected results:

1. Enter a valid delivery address. Expected: restaurants serving that address are displayed.

2. Choose an available restaurant and menu item. Expected: item details and price are displayed.

3. Add an available item to the cart. Expected: item, quantity and price appear correctly.

4. Place an order using a valid address and approved test payment details. Expected: order confirmation and order number are generated.

5. Open order tracking. Expected: the latest order status is displayed.

Conclusion / notes: Check menu prices, delivery charges, estimated delivery time and order status against the test data.

QUESTION 7: Food Delivery Application – Negative Test Scenarios

Requirement analysis: Test rejection and error handling for invalid addresses, unavailable items, empty carts and payment errors.

Test scenarios and expected results:

1. Enter an address outside the delivery area. Expected: a clear delivery-unavailable message appears.

2. Try to add an unavailable menu item. Expected: the application prevents ordering or marks it unavailable.

3. Proceed to checkout with an empty cart. Expected: checkout is blocked.

4. Use invalid payment test data. Expected: payment fails and the order is not shown as confirmed.

5. Enter incomplete or malformed contact details. Expected: field-level validation appears.

Conclusion / notes: Verify that failed orders do not create unintended charges and that cart contents remain consistent.

QUESTION 8: Banking Application – Positive Test Scenarios

Requirement analysis: Identify requirements for secure login, balance enquiry, account statement and fund transfer.

Test scenarios and expected results:

1. Log in with valid test credentials. Expected: account dashboard opens.

2. Request a balance enquiry. Expected: the correct test balance is displayed.

3. View a statement for a valid date range. Expected: matching transactions are listed.

4. Transfer a permitted amount to a valid test beneficiary. Expected: confirmation and transaction reference are generated.

Conclusion / notes: Use only the institution's approved test environment and test accounts. Check account and transaction data consistency.

QUESTION 9: Banking Application – Negative Test Scenarios

Requirement analysis: Test authentication, input validation and safe rejection of invalid transactions.

Test scenarios and expected results:

1. Enter incorrect login credentials. Expected: access is denied.

2. Enter zero, negative or over-limit transfer amount. Expected: transaction is rejected.

3. Select an invalid beneficiary. Expected: validation or rejection message appears.

4. Attempt a transaction after the session expires. Expected: re-authentication is required.

5. Submit the same transfer request again. Expected: duplicate transfer is prevented.

Conclusion / notes: Confirm that no unauthorized debit occurs and that error messages do not expose confidential account information.

QUESTION 10: Test Plan – Library Management System

Requirement analysis: Objective: test member management, catalogue search, book issue and return, renewals, reservations, fines and reports.

Test scenarios and expected results:

1. Testing scope: functional, usability, compatibility, security and basic performance testing.

2. Testing methods: requirement review, smoke testing, functional testing, regression testing and user acceptance testing.

3. Test design techniques: equivalence partitioning, boundary value analysis, decision tables and state-transition testing.

4. Test environment and data: test server, supported browsers, librarian/member accounts, sample books, due dates and controlled fine values.

5. Entry criteria: requirements reviewed, build deployed and test data ready. Exit criteria: planned tests executed, critical defects closed and test summary approved.

Conclusion / notes: Deliverables: test plan, test cases, test data, defect log, execution report and test summary. Risks include unstable environment, incomplete requirements and unavailable integrations.

QUESTION 11: Railway Reservation System – Positive Test Scenarios

Requirement analysis: Identify requirements for train search, seat availability, passenger details, booking and ticket confirmation.

Test scenarios and expected results:

1. Search using a valid source, destination and journey date. Expected: matching trains are displayed.

2. Select a train with available seats. Expected: fare and availability are displayed.

3. Enter valid passenger details and complete approved test payment. Expected: booking confirmation and PNR are generated.

4. Retrieve a booking using a valid PNR. Expected: correct journey details are displayed.

Conclusion / notes: Verify journey date, passenger count, fare and seat inventory using controlled test data.

QUESTION 12: Railway Reservation System – Negative Test Scenarios

Requirement analysis: Test invalid search details, passenger information and booking failures.

Test scenarios and expected results:

1. Enter the same source and destination or an invalid journey date. Expected: validation message appears.

2. Request more seats than allowed or available. Expected: booking is rejected.

3. Leave mandatory passenger fields blank. Expected: submission is blocked.

4. Simulate payment failure or timeout. Expected: booking is not falsely confirmed and seat inventory remains consistent.

5. Enter an invalid PNR. Expected: a safe booking-not-found message appears.

Conclusion / notes: Check that failed bookings do not create duplicate tickets or incorrectly reduce seat inventory.

QUESTION 13: Online Examination System – Positive Test Scenarios

Requirement analysis: Identify requirements for candidate login, instructions, exam access, question navigation, answer saving and submission.

Test scenarios and expected results:

1. Log in with valid candidate credentials during the permitted time. Expected: exam instructions are displayed.

2. Start an available examination. Expected: questions are displayed.

3. Select answers and move between questions. Expected: selected answers are retained.

4. Submit before the deadline. Expected: submission confirmation is displayed and responses are recorded.

Conclusion / notes: Verify timer behavior, autosave, allowed navigation and result calculation using approved test accounts.

QUESTION 14: Online Examination System – Negative Test Scenarios

Requirement analysis: Test access control, invalid inputs and exam-state handling.

Test scenarios and expected results:

1. Enter invalid login credentials. Expected: exam access is denied.

2. Try to start before the opening time or after the closing time. Expected: access is blocked.

3. Leave required fields unanswered. Expected: the configured warning or validation appears.

4. Try to reopen an already submitted exam. Expected: a duplicate attempt is prevented unless the rules allow it.

5. Simulate a network interruption during answer saving. Expected: recovery follows the system requirements.

Conclusion / notes: Check fairness, data integrity, clear error messages and audit records. Do not use real candidate data.

QUESTION 15: Test Plan – Inventory Management System

Requirement analysis: Objective: validate item master, stock receipt, issue/dispatch, stock adjustment, supplier records, reorder alerts and reports.

Test scenarios and expected results:

1. Scope: functional, integration, security and performance testing.

2. Test design: positive and negative cases, equivalence partitioning, boundary values for quantity and role-based access tests.

3. Execution strategy: smoke test after deployment, module-level functional tests, integration tests, regression after fixes and user acceptance testing.

4. Test data/environment: sample products, suppliers, warehouses, role-based users, opening stock and reorder thresholds.

5. Entry criteria: approved requirements and stable build. Completion criteria: critical tests pass, no open critical defects and test report is approved.

Conclusion / notes: Deliverables: test plan, scenarios, test cases, test data, defect report and closure summary. Risks include inaccurate opening stock, concurrent updates and third-party integration delays.

PART B – JAVA PROGRAMS AND EXECUTION (Q16–Q30)

Each question includes a Java method, a JUnit 5 assertion example and a sample result. The combined Java verification program was compiled and executed; the console output follows.

QUESTION 16: Check whether two strings are anagrams

Java program and JUnit 5 assertion:

```
static boolean isAnagram(String a, String b) {
    if (a == null || b == null) return false;
    char[] x = a.replaceAll("\\s+", "").toLowerCase().toCharArray();
    char[] y = b.replaceAll("\\s+", "").toLowerCase().toCharArray();
    java.util.Arrays.sort(x);
    java.util.Arrays.sort(y);
    return java.util.Arrays.equals(x, y);
}
// JUnit 5:
assertTrue(isAnagram("listen", "silent"));
```

Sample execution: Input: "listen", "silent" Expected: true Result: PASS

QUESTION 17: Find the maximum of three numbers

Java program and JUnit 5 assertion:

```
static int maximum(int a, int b, int c) {
    return Math.max(a, Math.max(b, c));
}
// JUnit 5:
assertEquals(27, maximum(12, 27, 19));
```

Sample execution: Input: 12, 27, 19 Output: 27 Result: PASS

QUESTION 18: Check whether a number is even or odd

Java program and JUnit 5 assertion:

```
static boolean isEven(int n) {
    return n % 2 == 0;
}
// JUnit 5:
assertTrue(isEven(24));
assertFalse(isEven(13));
```

Sample execution: Input: 24 and 13 Output: even and odd Result: PASS

QUESTION 19: Calculate compound interest using a function

Java program and JUnit 5 assertion:

```
static double compoundInterest(double p, double rate, int years) {
    double amount = p * Math.pow(1 + rate / 100.0, years);
    return amount - p;
}
// JUnit 5:
assertEquals(210.0, compoundInterest(1000, 10, 2), 0.001);
```

Sample execution: Principal = 1000; Rate = 10%; Years = 2 Interest = 210.00 PASS

QUESTION 20: Count vowels and consonants in a string

Java program and JUnit 5 assertion:

```
static int[] countVC(String s) {
    int vowels = 0, consonants = 0;
    for (char ch : s.toLowerCase().toCharArray()) {
        if (ch >= 'a' && ch <= 'z') {
            if ("aeiou".indexOf(ch) >= 0) vowels++;
            else consonants++;
        }
    }
    return new int[]{vowels, consonants};
}
// JUnit 5:
assertArrayEquals(new int[]{3, 7}, countVC("Hello World"));
```

Sample execution: Input: "Hello World" Vowels = 3; Consonants = 7 PASS

QUESTION 21: Check whether a number is prime

Java program and JUnit 5 assertion:

```
static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
// JUnit 5:
assertTrue(isPrime(29));
assertFalse(isPrime(21));
```

Sample execution: Input: 29 and 21 Output: prime and not prime PASS

QUESTION 22: Reverse a number

Java program and JUnit 5 assertion:

```
static int reverseNumber(int n) {
    int sign = n < 0 ? -1 : 1;
    int x = Math.abs(n), reverse = 0;
    while (x > 0) {
        reverse = reverse * 10 + x % 10;
        x = x / 10;
    }
    return sign * reverse;
}
// JUnit 5:
assertEquals(4321, reverseNumber(1234));
```

Sample execution: Input: 1234 Output: 4321 Result: PASS

QUESTION 23: Find the sum of digits of a number

Java program and JUnit 5 assertion:

```
static int sumDigits(int n) {
    int x = Math.abs(n), sum = 0;
    while (x > 0) {
        sum = sum + x % 10;
        x = x / 10;
    }
    return sum;
}
// JUnit 5:
assertEquals(16, sumDigits(583));
```

Sample execution: Input: 583 Calculation: 5 + 8 + 3 Output: 16 PASS

QUESTION 24: Compare two strings ignoring case sensitivity

Java program and JUnit 5 assertion:

```
static boolean equalIgnoreCase(String a, String b) {
    return a != null && b != null && a.equalsIgnoreCase(b);
}
// JUnit 5:
assertTrue(equalIgnoreCase("Software", "SOFTWARE"));
```

Sample execution: Input: "Software", "SOFTWARE" Output: true Result: PASS

QUESTION 25: Calculate a student's grade based on marks

Java program and JUnit 5 assertion:

```
static String grade(int marks) {
    if (marks < 0 || marks > 100) return "Invalid";
    if (marks >= 90) return "A";
    if (marks >= 80) return "B";
    if (marks >= 70) return "C";
    if (marks >= 60) return "D";
    return "F";
}
// JUnit 5:
assertEquals("B", grade(85));
```

Sample execution: Input: 85 marks Output: Grade B Result: PASS

QUESTION 26: Find the largest element in an array

Java program and JUnit 5 assertion:

```
static int largest(int[] a) {
    if (a == null || a.length == 0)
        throw new IllegalArgumentException("Array must not be empty");
    int max = a[0];
    for (int n : a) {
        if (n > max) max = n;
    }
    return max;
}
// JUnit 5:
assertEquals(19, largest(new int[]{4, 19, 7, 2}));
```

Sample execution: Input: [4, 19, 7, 2] Output: 19 Result: PASS

QUESTION 27: Calculate the average of n numbers

Java program and JUnit 5 assertion:

```
static double average(int[] a) {
    if (a == null || a.length == 0)
        throw new IllegalArgumentException("At least one number is required");
    long sum = 0;
    for (int n : a) sum += n;
    return (double) sum / a.length;
}
// JUnit 5:
assertEquals(20.0, average(new int[]{10, 20, 30}), 0.001);
```

Sample execution: Input: 10, 20, 30 Calculation: 60 / 3 Average = 20.00 PASS

QUESTION 28: Check whether a number is an Armstrong number

Java program and JUnit 5 assertion:

```
static boolean isArmstrong(int n) {
    if (n < 0) return false;
    String s = Integer.toString(n);
    int power = s.length(), sum = 0;
    for (char ch : s.toCharArray()) {
        sum += (int) Math.pow(ch - '0', power);
    }
    return sum == n;
}
// JUnit 5:
assertTrue(isArmstrong(153));
```

Sample execution: Input: 153 Calculation: 1^3 + 5^3 + 3^3 = 153 PASS

QUESTION 29: Convert temperature from Celsius to Fahrenheit

Java program and JUnit 5 assertion:

```
static double toFahrenheit(double celsius) {
    return celsius * 9.0 / 5.0 + 32;
}
// JUnit 5:
assertEquals(77.0, toFahrenheit(25), 0.001);
```

Sample execution: Input: 25°C Output: 77°F Result: PASS

QUESTION 30: Find string length without using a built-in length function

Java program and JUnit 5 assertion:

```
static int stringLength(String s) {
    if (s == null)
        throw new IllegalArgumentException("String must not be null");
    int count = 0;
    for (char ch : s.toCharArray()) {
        count++;
    }
    return count;
}
// JUnit 5:
assertEquals(7, stringLength("Testing"));
```

Sample execution: Input: "Testing" Output: 7 Result: PASS

ACTUAL JAVA EXECUTION OUTPUT

Compiled with javac and executed successfully:

```
PASS: Q16 anagram
PASS: Q17 maximum
PASS: Q18 even/odd
PASS: Q19 compound interest
PASS: Q20 vowels/consonants
PASS: Q21 prime
PASS: Q22 reverse number
PASS: Q23 sum of digits
PASS: Q24 string comparison
PASS: Q25 grade
PASS: Q26 largest array value
PASS: Q27 average
PASS: Q28 Armstrong
PASS: Q29 temperature conversion
PASS: Q30 string length
All 15 program checks passed.
```

Final result: All 15 program checks for Questions 16–30 passed. The combined program uses explicit runtime checks to display PASS/FAIL without an external JUnit dependency. JUnit 5 assertion examples can be run in a JUnit-enabled Java project.